

Railway Economic Simulation

Projekt portfolio — Roadmap & założenia v0.1

Checkpoint po sesji projektowej • 13 sierpnia 2026

Status dokumentu: roboczy punkt odniesienia, a nie zamknięta specyfikacja. Założenia oznaczone jako „Open Decision” pozostają celowo do dopracowania.

1. Cel projektu

- Stworzenie gry/symulatora ekonomicznego o zarządzaniu spółkami kolejowymi w USA w epoce rozwoju kolei, inspirowanej mechanikami serii 18XX.
- Projekt ma jednocześnie pełnić funkcję długoterminowego projektu portfolio dla ścieżki Data Analyst / szeroko rozumianego programowania.
- Gra ma być generatorem realistycznie wyglądających, kontekstowych danych biznesowych, które później można analizować.
- Docelowo projekt ma umożliwiać zarówno rozgrywkę człowieka, jak i automatyczne symulacje oraz eksperymenty z agentami AI.
- Najważniejsza zasada: najpierw problem i potrzeba projektu, dopiero potem technologia.

2. Wartość portfolio

Projekt ma pokazywać nie tylko znajomość narzędzi, ale cały sposób myślenia: projektowanie systemu → generowanie danych → testowanie → persystencja → SQL → EDA → wizualizacja → eksperymenty → ML/RL.

- Python i projektowanie modularnego systemu.
- Pytest i systematyczne testowanie logiki domenowej.
- Pandas + Jupyter + Matplotlib/Seaborn do EDA i analizy.
- CSV jako pierwszy etap persystencji, następnie migracja do SQL/PostgreSQL.
- Streamlit jako interfejs gry oraz późniejszy dashboard analityczny.
- Git/GitHub jako podstawowe narzędzie wersjonowania i dokumentowania rozwoju.
- Gymnasium, PyTorch oraz TensorBoard / Weights & Biases jako późniejsze rozszerzenie AI/RL.
- Teoria grafów i algorytm Dijkstry jako rzeczywisty element logiki transportu.
- Repository Pattern, MDP i EDA mają być poznawane wtedy, gdy pojawi się dla nich realny problem.

3. Wizja świata gry

- Mapa: historyczne USA z okresu świetności kolei.
- Mapa ma być stała, a odległości między miastami możliwie zgodne z rzeczywistością.
- Zamiast odwzorowywania każdej miejscowości początkowo preferowana jest ograniczona liczba strategicznych miast.
- Miasta posiadają różne charakterystyki gospodarcze i mogą pełnić kilka funkcji jednocześnie.
- Wstępne typy/cechy: rolnicze, przemysłowe, portowe, wydobywcze, duże centra ludności, centra handlowe/kolejowe oraz finansowe.
- Sieć torów jest częścią świata i może być wykorzystywana przez wiele spółek.

4. Towary i gospodarka

- Towary mają odpowiadać realiom epoki; główne kategorie to żywność, materiały budowlane i produkty przemysłowe, z możliwością dodania surowców oraz importów.
- Miasta produkują i konsumują towary niezależnie od graczy.
- Produkcja nie jest sztywnym automatem: miasto/fabryka ma reagować na popyt, ceny, zapasy, dostępność surowców i opłacalność.
- Miasto nie powinno produkować bez końca dóbr, których nikt nie potrzebuje.
- Potrzeby mają mieć różną ważność — np. żywność i opał powinny mieć wyższy priorytet niż dobra luksusowe.
- Występują lokalne rynki: brak łatwego dostępu do towaru oraz wysoki popyt powodują wzrost ceny.
- Duża dostawa jednego przewoźnika może zaspokoić popyt i obniżyć cenę dla kolejnych spółek.
- Sezonowość jest pożądanym elementem systemu.
- Losowe wydarzenia mogą zmieniać podaż, popyt lub możliwości transportowe.

5. Wstępny model ceny towaru

Cena powinna być interpretowalna, a nie „magiczna”. Wstępna koncepcja: $\text{cena bazowa} \times \text{czynnik niedoboru} \times \text{sezonowość} \times \text{wydarzenia} \times \text{warunki lokalne}$. Kluczowym mechanizmem ma być relacja aktualnego zapasu do docelowego poziomu zapasów.

Open Decision: konkretny wzór matematyczny, dynamika reakcji cen oraz parametry modelu zostaną ustalone po dalszym projektowaniu i testach symulacji.

6. Spółki kolejowe

- Docelowo około 10 aktywnych spółek, ale pierwsza wersja powinna być znacznie prostsza.
- Spółki początkowo pozostają uśpione i mogą zostać uruchomione, gdy pojawi się sponsor/prezes — inspiracja 18XX.
- Pieniądze spółki i osobisty majątek biznesmena są rozdzielone.
- Spółka kupuje pociągi i wagony za własne środki.
- Spółka może wypłacać dywidendę lub zatrzymywać zysk na rozwój.
- Maksymalna pula akcji roboczo: 1000 akcji na spółkę.
- Prezes kontroluje spółkę przez posiadanie większości akcji.
- Przejęcie kontroli jest możliwe poprzez zgromadzenie największego udziału.
- Prezes projektuje i edytuje trasy handlowe zarządzanej spółki.

7. Pociągi, tory i trasy

- Pociągi mają ograniczoną liczbę wagonów zależną od klasy lokomotywy.
- Fizyczna sieć torów jest stała.
- Trasa handlowa jest decyzją spółki wykorzystującą istniejącą sieć.
- Prezes wskazuje miasto początkowe, docelowe oraz konkretny towar do zakupu i sprzedaży.
- Trasy mogą mieć powrót z innym towarem, aby ograniczać puste przebiegi.
- Możliwe są pętle wieloetapowe.
- Awarie mogą blokować fragment sieci, zmuszając pociąg do znalezienia alternatywnej ścieżki.
- Dijkstra ma być wykorzystana do wyszukiwania tras w grafie.

8. Giełda i mali inwestorzy

Giełda ma żyć własnym życiem. Cena akcji ma wynikać z połączenia fundamentów spółki oraz rzeczywistego popytu/podaży generowanego przez graczy i NPC.

- Wartość fundamentalna ma uwzględniać m.in. wyniki, aktywa, zyski i perspektywy spółki.
- Rynek może odchyłać cenę od wartości fundamentalnej.
- Dywidendy, reinwestycje i wyniki spółki wpływają na zachowanie inwestorów.
- Małe grupy/NPC inwestorów mają różne preferencje i strategie.
- Wstępne typy: inwestor dywidendowy, wzrostowy, momentum, value oraz losowy.
- NPC nie muszą być od początku zaawansowanymi agentami ML — proste reguły wystarczą.
- Zachowania NPC mają generować naturalne dane o transakcjach, cenach i reakcjach rynku.

9. Przebieg tury

Podstawowa jednostka czasu: 1 tura = 1 dzień. Gracz wykonuje dostępne akcje, a po zakończeniu tury świat wykonuje automatyczne przejście do kolejnego stanu.

- Faza decyzji gracza: akcje, akcje giełdowe, trasy, pociągi, wagony i inne dostępne decyzje.
- Faza decyzji pozostałych uczestników/NPC.
- Ruch pociągów.
- Sprzedaż/dostawa towarów po dotarciu do celu.
- Produkcja miast.
- Konsumpcja i zużycie zapasów.
- Aktualizacja rynku i cen.
- Wydarzenia losowe.
- Księgowanie wyników dnia.
- Co około 31 dni: rozliczenie miesięczne i decyzje dotyczące dywidend.

Open Decision: dokładna kolejność operacji w ramach dnia, moment występowania eventów oraz szczegółowe reguły rozliczeń zostaną doprecyzowane przed implementacją Game Engine.

10. Warunek zwycięstwa

Największy majątek netto na koniec rozgrywki. Wstępnie: gotówka + rynkowa wartość posiadanych akcji + inne aktywa.

Wartość akcji ma być liczona po aktualnej cenie rynkowej, a nie po cenie zakupu.

11. Architektura — zasady nadrzędne

- Logika gry ma działać w czystym Pythonie i być niezależna od interfejsu.
- Streamlit nie powinien zawierać logiki domenowej; ma wywoływać Game Engine.
- Ta sama logika ma być możliwa do wykorzystania przez człowieka, testy, automatyczne symulacje i przyszłych agentów.
- Warstwa danych ma być oddzielona od logiki gry.
- CSV jest pierwszym backendem danych; później możliwa jest migracja do PostgreSQL/SQL.
- Repository Pattern ma zostać zastosowany wtedy, gdy pojawi się rzeczywista potrzeba oddzielenia sposobu przechowywania danych.
- Projekt ma od początku mieć sensowną strukturę katalogów, modułów i testów.
- Nowe technologie nie są celem samym w sobie — każda ma rozwiązywać realny problem projektu.

12. Dane i logowanie

- Gra ma być generatorem dużych zbiorów danych o charakterze biznesowym.
- Preferowane jest logowanie zdarzeń, a nie tylko końcowego stanu gry.
- Robocze typy zdarzeń: start gry, zakup akcji, zakup pociągu, załadowanie towaru, dostawa, zmiana ceny, dywidenda, koniec gry itd.
- Dane powinny umożliwiać analizę przyczynową/historiczną: co doprowadziło do sukcesu, porażki, bankructwa lub zmiany ceny.
- W późniejszym etapie dane mają trafić do relacyjnej bazy SQL.

13. Analityka

- Jupyter Notebook jako środowisko eksploracji danych.
- Pandas do transformacji i analizy.
- Matplotlib/Seaborn do wizualizacji.
- EDA ma odpowiadać na pytania biznesowe, a nie być tylko kolekcją wykresów.
- Przykładowe pytania: rentowność tras, zależność ceny od podaży, ROI, zachowanie inwestorów, wykorzystanie floty, sezonowość, skuteczność prognoz opłacalności.
- Docelowo Streamlit może prezentować zarówno grę, jak i dashboard analityczny.

14. AI / boty — docelowa ścieżka

Boty mają być projektowane jako naturalne rozszerzenie gry, a nie osobny dodatek. Każda decyzja gracza powinna być potencjalnie reprezentowalna jako Action, a stan świata jako State.

- Poziom 0: losowe decyzje.
- Poziom 1: proste reguły.
- Poziom 2: heurystyczni agenci.
- Poziom 3: modele ML/predykcja.
- Poziom 4: reinforcement learning.
- Gymnasium jako środowisko RL.
- PyTorch jako narzędzie ML.
- TensorBoard / Weights & Biases do śledzenia eksperymentów.
- MDP ma zostać poznane i zastosowane wtedy, gdy pojawi się problem decyzyjny wymagający formalizacji.

15. Testowanie

- Pytest od bardzo wczesnego etapu projektu.
- Testowanie ma obejmować małe reguły domenowe, np. zakup pociągu, transport, przychód, dywidendy i zmianę zapasów.
- Testy mają chronić przed regresjami w miarę rozrostu symulacji.
- Każdy milestone powinien mieć własne kryteria Definition of Done.

16. Git i GitHub

- Git/GitHub od pierwszego dnia projektu.
- Regularne, sensowne commity opisujące rozwój.
- README oraz dokumentacja decyzji architektonicznych.
- Projekt ma być rozwijany jako publiczne, prezentowalne portfolio, a nie tylko prywatny folder.
- Docelowo dokumentować również eksperymenty i uzasadnienie decyzji.

17. Wstępna roadmapa

Etap	Cel / checkpoint
0. Fundament	Git/GitHub, środowisko, struktura projektu, pierwsze testy.
1. Minimalny Game Engine	Kilka miast, jedna spółka, pociąg, towary, podstawowy transport. Można rozegrać dzień/turę.
2. Ekonomia	Produkcja, konsumpcja, zapasy, ceny, koszty, przychody, dywidendy, akcje.
3. Pełniejszy świat	Więcej miast, sieć kolejowa, trasy, awarie, sezonowość, wydarzenia.
4. Data layer	Logowanie zdarzeń, CSV, spójny model danych.
5. Analytics	Pandas, Jupyter, EDA, wizualizacje i pytania biznesowe.
6. SQL	Migracja warstwy persystencji do PostgreSQL / analiza SQL.
7. Streamlit	Grywalny interfejs + dashboard analityczny.
8. NPC / heurystyki	Mali inwestorzy i proste boty gospodarcze.
9. ML / RL	Gymnasium, PyTorch, eksperymenty agentów i porównania strategii.

18. Zasada checkpointów

Projekt nie ma czekać do końca, aby stać się portfolio. Każdy większy milestone powinien pozostawiać działający, prezentowalny artefakt.

- Checkpoint 1: działający minimalny silnik gry.
- Checkpoint 2: działająca uproszczona gra ekonomiczna.
- Checkpoint 3: dane + pierwsza analiza.
- Checkpoint 4: SQL + dashboard.
- Checkpoint 5: boty/NPC.
- Checkpoint 6: eksperymenty ML/RL.

19. Sposób współpracy — ustalenie

Użytkownik jest głównym wykonawcą projektu. Samodzielnie projektuje, implementuje, testuje i poznaje kod. Asystent pełni rolę mentora / project managera / tech leada.

- Asystent pilnuje kierunku, zakresu, architektury, zależności i milestone'ów.
- Asystent tłumaczy nowe koncepty wtedy, gdy stają się potrzebne.
- Przy problemach preferowane są najpierw pytania i wskazówki, a nie gotowy kod.
- Domyślna drabina pomocy: hint → mocniejsza odpowiedź → pseudokod/design → przykład kodu.
- Code review ma skupiać się na jakości, czytelności, testach, architekturze i przyszłej rozszerzalności.
- Użytkownik powinien rozumieć każdą istotną część projektu i być w stanie wyjaśnić decyzje podczas rozmowy rekrutacyjnej.
- Roadmapa jest żywa: może się zmieniać, jeśli nowe informacje lub eksperymenty pokażą lepszy kierunek.

20. Zasady, których pilnujemy

- Nie budujemy wszystkiego naraz.
- Nie zaczynamy od zaawansowanego AI.
- Nie overengineerujemy architektury przed pojawieniem się problemu.
- Nie dodajemy technologii wyłącznie dla CV.
- Najpierw mały, działający system; potem rozszerzenia.
- Każda warstwa powinna być możliwa do wykorzystania niezależnie.
- Projekt ma być wystarczająco ciekawy, żeby chciało się go rozwijać przez długi czas.
- Fun z gry jest pełnoprawnym wymaganiem — projekt ma motywować do dalszej nauki.

21. Open Decisions — do dalszego projektowania

- Dokładny model produkcji miast i reakcji producentów na opłacalność.
- Dokładny wzór ceny towaru i parametry reakcji na niedobór/nadmiar.
- Hierarchia potrzeb oraz mechanika konsumpcji.
- Dokładna lista towarów i ich historyczna reprezentacja.
- Lista miast i dokładna sieć kolejowa.
- Dokładna mechanika emisji/skupu akcji.
- Dokładny model wartości fundamentalnej i mechanizmu giełdowego.
- Dokładne strategie oraz liczba NPC inwestorów.
- Dokładna kolejność operacji w pojedynczym dniu.

- Warunki końca gry poza samym rankingiem majątku.
- Dokładna reprezentacja stanu, akcji i nagrody dla przyszłego środowiska RL.

22. Następny krok

Najbliższym zadaniem projektowym jest zaprojektowanie jednego pełnego dnia rozgrywki: co dzieje się na początku dnia, jakie akcje są dostępne, jak działają decyzje NPC, jak poruszają się pociągi, kiedy zachodzą transakcje towarowe, produkcja, konsumpcja, aktualizacja cen, eventy i rozliczenia. Dopiero na tej podstawie będziemy wyprowadzać Game State, Actions, Events, Turn Engine, logowanie danych, testy i późniejszą strukturę środowiska Gymnasium.

Wersja: v0.1 — checkpoint po pierwszej sesji projektowej.

Charakter: living document — będzie aktualizowany wraz z rozwojem projektu.